Ordenação pelo método Heapsort

Considere o seguinte vetor de caracteres:

A = { 'M', 'U', 'L', 'T', 'I', 'P', ‘R', 'O', 'C', 'E', ‘S', ‘S', 'A', ‘D',‘O' }

(A) [0,6] Execute `CONSTRÓI-MAX-HEAP(A, n)` sobre o vetor apresentado. Represente graficamente o heap de máximo obtido, também indicando o vetor correspondente. Explique qual(is) propriedade(s) caracteriza(m) um heap de máximo.

Tendo o vetor inicial de caracteres: A = { 'M', 'U', 'L', 'T', 'I', 'P', ‘R', 'O', 'C', 'E', ‘S', ‘S', 'A', ‘D',‘O' }

temos que achar o indice inicial, temos 15 elementos, portanto usamos a formula:

indice = n/2
indice = 15/2
indice= 7

Com isso iremos do indice 7 até o indice 1

após isso vamos aplicar o Constrói Max-Heap

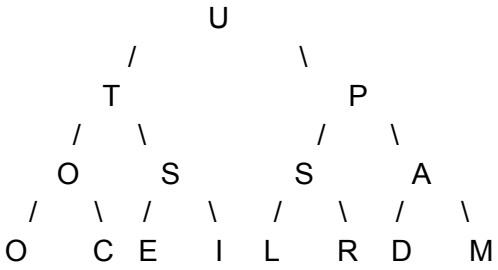
- Filho esquerdo: $2i$
- Filho direito: $2i + 1$
- O maior elemento deve ficar no nó pai.

Agora Iremos realizar as comparações

Índice	Comparação	Resultado
7	R, D, O	Troca R com O
6	S, S, A	Não troca
5	I, E, S	Troca I com S
4	T, O, C	Não troca
3	L, P, O	Troca L com P
2	U, T, S	Não troca
1	M, U, P	Troca M com U

Assim temos o Max Heap: {U,T,P,O,S,S,A,O,C,E,I,L,R,D,M}

Podemos representar graficamente por:



Um Max-Heap possui duas propriedades:

- É uma árvore binária completa: os níveis são preenchidos da esquerda para a direita.
- O pai sempre possui valor maior ou igual aos seus filhos.

(B) [0,6] Considerando o heap obtido no item A, execute a primeira iteração do laço principal do Heapsort (i=n), incluindo a *troca* dos elementos, a *redução* do tamanho do heap e a *chamada* de `MAXIMIZA-HEAP(A, 1)`. Representa o estado final do vetor e do heap, identificando a região do vetor que já está ordenada. Explique por que a chamada de `MAXIMIZA-HEAP` é necessária.

Tendo o heap {U,T,P,O,S,S,A,O,C,E,I,L,R,D,M}

O Heapsort começa trocando a raiz (maior elemento) com o último elemento do vetor.

- Primeiro elemento: U
- Último elemento: M

Após a troca:

$$A = \{M, T, P, O, S, S, A, O, C, E, I, L, R, D, U\}$$

Agora nosso heap tem uma unidade a menos
ficando com 15-1=14 e sem o elemento U

Ao executar o MAXIMIZA HEAP

Agora precisamos reorganizar o heap, pois M está na raiz.

Vamos comparar M com seus filhos:

Primeira comparação:

- Pai: M
- Filho esquerdo: T
- Filho direito: P

T é o maior. Portanto, trocamos M com T.

$$\{T, M, P, O, S, S, A, O, C, E, I, L, R, D, U\}$$

Segunda comparação:

Agora M está na posição 2.

- Pai: M
- Filho esquerdo: O
- Filho direito: S

S é o maior. Trocamos M com S.

$$\{T, S, P, O, M, S, A, O, C, E, I, L, R, D, U\}$$

Terceira comparação:

Agora M está na posição 5.

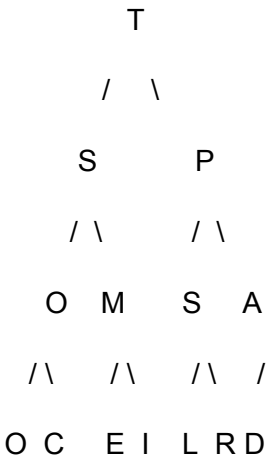
- Pai: M
- Filho esquerdo: E
- Filho direito: I

M é maior que E e I. Portanto, não precisamos realizar outra troca.

o Vetor final vira:

A= {T,S,P,O,M,S,A,O,C,E,I,L,R,D,U}

e Representaçao grafica do heap



Por que MAXIMIZA-HEAP é necessária?

A troca da raiz com o último elemento pode fazer com que a propriedade de Max-Heap seja violada.

Por isso, chamamos MAXIMIZA-HEAP para reorganizar a árvore, garantindo que o maior elemento restante volte para a raiz.

Assim, ele poderá ser colocado na próxima posição definitiva durante a próxima iteração do Heapsort.

Na primeira iteração, troca-se a raiz U com o último elemento M. Em seguida, o tamanho do heap é reduzido de 15 para 14, deixando U na região ordenada. Depois, executa-se MAXIMIZA-HEAP(A,1), realizando as trocas M com T e, posteriormente, M com S. O vetor resultante é `{T,S,P,O,M,S,A,O,C,E,I,L,R,D,U}`. A chamada de MAXIMIZA-HEAP é necessária para restaurar a propriedade de Max-Heap.

(C) [0,3] Assinale a alternativa correta sobre o Heapsort:

- ( ) É estável e possui tempo de execução O(n log n) no pior caso.
- (X) Não é estável e possui tempo de execução O(n log n) no pior caso.
- ( ) É estável e possui tempo de execução $O(n^2)$ no pior caso.
- ( ) Não é estável e necessita de espaço auxiliar O(n) na implementação usual *in-place* (local ou *in loco*).

```
MAXIMIZA-HEAP(A, i)
1       l = ESQUERDA(i)
2       r = DIREITA(i)
3       if l ≤ A.tamanho-do-heap e A[l] > A[i]
4               maior = l
5       else   maior = i
6       if r ≤ A.tamanho-do-heap e A[r] > A[maior]
7               maior = r
8       if maior ≠ i
9               trocar A[i] com A[maior]
10              MAXIMIZA-HEAP(A, maior)

CONSTRÓI-MAX-HEAP(A, n)
1       A.tamanho-do-heap = n
2       for i = ⌊ n/2 ⌋ downto 1
3               MAXIMIZA-HEAP(A, i)


HEAPSORT(A, n)

1       CONSTRÓI-MAX-HEAP(A, n)
2       for i = n downto 2
3               trocar A[1] com A[i]
4               A.tamanho-do-heap = A.tamanho-do-heap – 1
5               MAXIMIZA-HEAP(A, 1)
```